

PawBook — RBAC & Permission Matrix

Overview

This document defines:

- Authentication flow
- Authorization (RBAC)
- Public vs protected routes
- Role permissions
- Ownership rules
- Middleware flow

Roles used in the system:

```
PUBLIC    = unauthenticated visitor
CLIENT   = authenticated customer
GROOMER  = authenticated groomer
ADMIN    = administrator
```

1. Authentication Flow

```
User logs in
  ↓
Backend validates credentials
  ↓
JWT token is generated
  ↓
Frontend stores token
  ↓
Frontend sends:
Authorization: Bearer <token>
  ↓
authenticate middleware verifies token
  ↓
req.user = {
  id,
  role
}
  ↓
authorize(...roles) checks permissions
```

↓
Controller executes

2. Middleware Architecture

authenticate middleware

Purpose:

- verifies JWT token
- identifies authenticated user
- attaches user to request

Result

```
req.user = {  
  id,  
  role  
}
```

Errors

Situation	Status
Missing token	401
Invalid token	401
Expired token	401

authorize(...roles) middleware

Purpose:

- checks whether authenticated user has required role

Example

```
authorize("ADMIN")  
authorize("CLIENT", "ADMIN")
```

Errors

Situation	Status
Not authenticated	401
Insufficient role	403

3. Route Middleware Order

Correct order:

```
asyncHandler
  ↓
authenticate
  ↓
authorize
  ↓
controller
```

Example:

```
router.get(
  "/admin-test",
  authenticate,
  authorize("ADMIN"),
  asyncHandler(controller.adminTest)
);
```

4. Permission Matrix

Resource / Action	PUBLIC	CLIENT	GROOMER	ADMIN
POST /auth/register	✓	—	—	—
POST /auth/login	✓	—	—	—
GET /auth/me	—	✓	✓	✓
POST /auth/logout	—	✓	✓	✓
GET /salon-info	✓	—	—	—

Resource / Action	PUBLIC	CLIENT	GROOMER	ADMIN
GET /salon-opening-hours	✓	—	—	—
GET /groomer-profiles	✓	—	—	—
GET /services	✓	—	—	—
Dogs CRUD	—	own only	—	✓
Bookings create/cancel	—	own	own schedule	✓
Groomer working hours	—	—	own	✓
Groomer time blocks	—	—	own	✓
Services manage	—	—	own	✓
POST /admin/groomers	—	—	—	✓
Update salon info	—	—	—	✓
Admin master calendar	—	—	—	✓

5. Ownership Rules

RBAC alone is NOT enough.

The system must also validate resource ownership.

Dog ownership

Rule:

```
Dog.customer_profile.userId === req.user.id
```

Meaning:

- CLIENT can only access their own dogs
- ADMIN can bypass ownership

Booking ownership (customer)

Rule:

```
Booking.customer_profile.userId === req.user.id
```

Meaning:

- CLIENT sees only own bookings
-

Booking ownership (groomer)

Rule:

```
Booking.groomer_profile.userId === req.user.id
```

Meaning:

- GROOMER sees only own schedule
-

Groomer resource ownership

Rules:

```
Service.groomer_profile.userId === req.user.id  
WorkingHours.groomer_profile.userId === req.user.id  
TimeBlock.groomer_profile.userId === req.user.id
```

Meaning:

- GROOMER manages only own resources
 - ADMIN can bypass ownership
-

6. PRD Security Rules

Public registration

```
POST /auth/register
```

Must create ONLY:

CLIENT accounts

Public registration MUST NOT allow:

ADMIN
GROOMER

Groomer creation

Only ADMIN can create groomers.

Example:

POST /admin/groomers

Admin accounts

Admin accounts are:

- seeded manually
- created internally
- never publicly registered

7. Recommended Architecture

```
src/  
├─ middleware/  
│   ├── authenticate.js  
│   └── authorize.js  
├─ config/  
│   └── permissions.js  
└─ routes/  
    ├── auth.routes.js  
    ├── dogs.routes.js  
    └── bookings.routes.js
```

```
|
|─ controllers/
|─ services/
|─ repositories/
|─ utils/
|   └─ httpError.js
```

8. Example Protected Routes

CLIENT only

```
router.use(
  "/dogs",
  authenticate,
  authorize("CLIENT", "ADMIN")
);
```

GROOMER only

```
router.use(
  "/groomer-working-hours",
  authenticate,
  authorize("GROOMER", "ADMIN")
);
```

ADMIN only

```
router.post(
  "/admin/groomers",
  authenticate,
  authorize("ADMIN")
);
```

9. Current Backend Status

Completed

- JWT authentication
- authenticate middleware
- authorize middleware
- HttpError handling
- Role enums
- Protected route support

Next Steps

1. Apply RBAC on CRUD routes
2. Add ownership validation
3. Implement booking authorization rules
4. Implement booking availability logic
5. Add booking overlap prevention